

ENSF-607

Advanced Software Design and Architecture

Project Proposal

SmartFix: Reporting and Repairing Public Infrastructure

Group 6

Team Members

Ashwin Shanmugam	30300738
Jasneet Singh	30044332
Devarsh Patel	30301526
Hareem Khan	30140959
Ranjit Singh	30301717
Peter Osaade	30279322
Sodeeq Alli	30296735

SmartFix: Reporting and Repairing Public Infrastructure

1. Project Idea

SmartFix is a web application for reporting and fixing damaged public infrastructure. A resident who spots a pothole, a broken streetlight, a water leak or a cracked sidewalk takes a photo, drops a pin on the map and submits it. SmartFix then checks the report, works out how serious it is, assigns it to a repair crew and keeps the resident updated until the repair is done.

We selected **SDG 9, Industry, Innovation and Infrastructure**, specifically target 9.1: reliable, resilient infrastructure that everyone can access fairly. Cities often hear about damage late, get the same problem reported many times, and fix things in the order they arrive rather than by how dangerous they are. Residents rarely hear back. SmartFix tackles this through four behaviours:

- Each new report is classified and given a severity score within seconds, so a dangerous problem goes to a crew ahead of a cosmetic one.
- When several people report the same problem, their reports are merged into one issue and its priority goes up. Crews are never sent to the same pothole twice.
- Extra priority from repeat reports is capped, and every open issue gains priority the longer it waits. Neighbourhoods where fewer people report problems still get their repairs.
- A repair that runs past its deadline is escalated automatically. An issue only closes once the resident confirms the fix, or after seven days with no reply.

SmartFix has three kinds of users. **Residents** submit reports, follow their progress and confirm or dispute a repair. **City operators** watch a live map of open issues, check reports the system was unsure about and reassign work when needed. **Repair crews** receive work orders, update their progress and upload a photo when the job is done.

2. Event-Driven Architecture

The backend is one FastAPI application split into six modules: Reports, Validation, Priority, Work Orders, Notifications and Analytics. The modules never call each other. When a module finishes its part of the job, it publishes an event to RabbitMQ, and every module that needs that event picks it up and does its own work.

Event producers: the Reports module (resident actions), the crew portal (crew actions), the Validation, Priority and Work Orders modules (when they finish processing), and a deadline scheduler that checks open work orders every few minutes.

Event channel: RabbitMQ. Every event has a name such as issue.reported and goes to a single exchange. Each consuming module has its own queue that only receives the events it subscribes to. If one module is slow or goes down, its events wait in its queue while the others keep working. Events that keep failing are moved to a separate queue so we can look at them.

Event	Producer	Consumers	What happens
IssueReported	Reports module, when a resident submits	Validation	The model identifies the problem and rates severity from 1 to 5. If an open issue of the same type is within 50 m, the model compares the two reports to decide if they match.
IssueValidated	Validation	Priority, Notifications	A priority score is calculated. The resident is told the report was received and what was found.
IssueDuplicateDetected	Validation	Priority, Notifications	The report joins the existing issue and its priority is recalculated. The resident is told it is already being tracked.
IssueNeedsReview	Validation, when the model is unsure or the photo is not infrastructure	Operator dashboard	The report goes to a review list. The operator approves it (publishing IssueValidated) or rejects it.
IssuePrioritized	Priority	Work Orders, operator dashboard	A work order is created or moved up or down the queue. The map updates.
WorkOrderCreated	Work Orders, after matching a crew by problem type, area and workload	Notifications, dashboard, deadline scheduler	The crew is notified. A deadline is set based on severity.
WorkOrderStatusChanged	Crew portal	Notifications, dashboard	The resident and operator see that work has started.
RepairCompleted	Crew portal, when the after photo is uploaded	Validation, Notifications, Analytics	The vision model compares before and after photos. The resident is asked to confirm. Repair time is recorded.
RepairDisputed	Reports module, when a resident says it is not fixed	Priority, Work Orders	The issue is reopened with higher priority and a new work order.
DeadlineMissed	Deadline scheduler	Priority, Notifications	The issue moves to the top of the queue and a supervisor is alerted.

How events change the system - Every issue moves through a fixed set of statuses: Reported, Validated, Prioritized, Assigned, In Progress, Completed and Closed, plus Reopened and Escalated. Each event moves an issue one status forward, and only the module that owns that step can change it. Every event is also saved to an events table in PostgreSQL, which gives operators the full history of each issue. If RabbitMQ delivers the same event twice, the module recognises its ID and skips it. The Notifications module uses the same local model to turn each event into a short message a resident can understand and pushes it to their browser straight away.

3. Technology Stack

Layer	Technology	How we use it
Frontend	React with TypeScript, Vite, Tailwind CSS, React-Leaflet with OpenStreetMap	Resident reporting page, crew portal and operator map.
Backend	Python, FastAPI, Pydantic	One application with six modules. Pydantic checks that every event carries the fields it should.
Event broker	RabbitMQ (free hosted instance on CloudAMQP), aio-pika for Python	Carries every event between modules. Its web console lets us show events moving during the demo.
Database	PostgreSQL with the PostGIS extension, hosted on Supabase	Our relational database. Tables are created with SQL scripts and all reads and writes use SQL. PostGIS finds reports close to each other.
Local AI	Ollama runs Qwen3-VL 8B, a single model that reads photos and text. Gemma 3 4B as a smaller fallback	Classifies photos, compares reports to catch duplicates, checks repairs and writes messages to residents. No paid API, and photos are never sent to an outside AI company.
Live updates	FastAPI WebSockets	The map and status pages update as soon as an event is handled.
Deployment	Frontend on Vercel. Backend and Ollama on a team member's machine, reachable online through Cloudflare Tunnel	Vercel only handles short requests, so it cannot keep RabbitMQ listeners or the language model running.
Testing and tools	GitHub, Pytest, Playwright	Tests run on every pull request, including end-to-end tests that submit a report and follow it to completion.

Scope - We will build photo classification, duplicate detection and generated notifications first. If time allows, we will add a map layer that highlights areas with frequent repeat problems. Predicting future repairs from past data is a stretch goal.

4. Team Roles

Member	Primary role	Responsibilities
Ranjit Singh	Software Project Manager / Software Architect	Sprint planning, task board, milestones, integration checkpoints and final deliverables.
Devarsh Patel	Test Engineer	Event list and formats, module boundaries, RabbitMQ setup, architecture and UML diagrams.
Ashwin Shanmugam	AI Engineer / Backend Engineer	Validation module, Ollama model setup, duplicate detection, prompts and model accuracy testing.
Jasneet Singh	Database Engineer	SQL schema and scripts, PostGIS location queries, events history table.
Peter Osaade	Backend Engineer	Reports, Priority, Work Orders and Notifications modules, deadline scheduler.
Sodeeq Alli	Frontend Engineer	React app, resident and crew pages, live operator map.
Hareem Khan	Requirements Analyst and Test Engineer	Requirements and use cases, test plan, CI pipeline, integration and end-to-end tests.